



Assignment 1 — BoW and Inverted Index

Corpus: Cranfield Collection (1,400 aerodynamics research abstracts)

Library: `ir_datasets`

Table of Contents

1. [Part \(a\) — Preprocessing & Vocabulary Analysis](#)
2. [Part \(b\) — Bag-of-Words: Unigram vs Bigram](#)
3. [Part \(c\) — Inverted Index with 3 Data Structures](#)
4. [Part \(d\) — Boolean Query Processing & Optimisation](#)
5. [Summary & Conclusion](#)

Imports & Setup

```
In [ ]: import sys, os, time, warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import ir_datasets

# Our custom modules
from preprocessing import (
    run_pipeline, PIPELINE_STEPS, PIPELINE_STEPS_LEMMA, preprocess_text,
)
from bow_model import BagOfWords
from inverted_index import (
    DictInvertedIndex, SortedListInvertedIndex, BSTInvertedIndex, benchmark_ir
)
from boolean_query import (
    run_boolean_query, run_boolean_query_optimized, run_boolean_query_ordered,
    complexity_analysis
)

sns.set_style('whitegrid')
plt.rcParams['figure.dpi'] = 120

print('All modules loaded successfully.')
```

All modules loaded successfully.

```
In [2]: # — Load the Cranfield Corpus —————
dataset = ir_datasets.load('cranfield')

doc_ids = []
doc_texts = []
doc_titles = []

for doc in dataset.docs_iter():
    doc_ids.append(doc.doc_id)
    doc_texts.append(doc.text)
    doc_titles.append(doc.title)

print(f'Loaded {len(doc_texts)} documents from the Cranfield collection.')
print(f'\nSample document (doc 0):')
print(f'  Title : {doc_titles[0]}')
print(f'  Text  : {doc_texts[0][:300]}...')
```

```
[INFO] [starting] http://ir.dcs.gla.ac.uk/resources/test_collections/cran/cran.tar.gz
[INFO] [finished] http://ir.dcs.gla.ac.uk/resources/test_collections/cran/cran.tar.gz: [00:00] [507kB] [689kB/s]
```

Loaded 1400 documents from the Cranfield collection.

Sample document (doc 0):

Title : experimental investigation of the aerodynamics of a wing in a slipstream .

Text : experimental investigation of the aerodynamics of a wing in a slipstream .

an experimental study of a wing in a propeller slipstream was made in order to determine the spanwise distribution of the lift increase due to slipstream at different angles of attack of the wing and at different free strea...

Part (a): Preprocessing & Vocabulary Analysis (20 marks)

We apply the following preprocessing pipeline step-by-step and observe the change in **total tokens** and **vocabulary size** at each stage:

1. Raw tokenization (NLTK `word_tokenize`)
2. Lowercasing
3. Punctuation / special-character removal
4. Stop-word removal (NLTK English stop-words)
5. Stemming (Porter Stemmer) **or** Lemmatization (WordNet)

Part (a): Run Stemming Pipeline

```
In [ ]: print('='*80)
print('PREPROCESSING PIPELINE – STEMMING PATH')
print('='*80)
stats_stem, processed_docs_stem = run_pipeline(doc_texts, PIPELINE_STEPS, verb

print('\n' + '='*80)
print('PREPROCESSING PIPELINE – LEMMATIZATION PATH')
print('='*80)
stats_lemma, processed_docs_lemma = run_pipeline(doc_texts, PIPELINE_STEPS_LEM
```

```
=====
=
PREPROCESSING PIPELINE – STEMMING PATH
=====
=
1. Raw Tokenization          | tokens= 239,132 | vocab= 9,751 | t
ime=0.6068s
2. Lowercasing              | tokens= 239,132 | vocab= 9,751 | t
ime=0.0170s
3. Punctuation Removal     | tokens= 220,447 | vocab= 9,733 | t
ime=0.0813s
4. Stop-word Removal       | tokens= 127,429 | vocab= 9,615 | t
ime=0.0080s
5. Stemming (Porter)       | tokens= 127,429 | vocab= 6,984 | t
ime=0.8075s

=====
=
PREPROCESSING PIPELINE – LEMMATIZATION PATH
=====
=
1. Raw Tokenization          | tokens= 239,132 | vocab= 9,751 | t
ime=0.5696s
2. Lowercasing              | tokens= 239,132 | vocab= 9,751 | t
ime=0.0184s
3. Punctuation Removal     | tokens= 220,447 | vocab= 9,733 | t
ime=0.0790s
4. Stop-word Removal       | tokens= 127,429 | vocab= 9,615 | t
ime=0.0080s
5. Lemmatization (WordNet) | tokens= 127,429 | vocab= 8,807 | t
ime=1.7267s
```

Part (a): Tabular comparison

```
In [ ]: rows = []
for step_name, vals in stats_stem.items():
    rows.append({
        'Step': step_name,
        'Total Tokens': vals['total_tokens'],
        'Vocab Size': vals['vocab_size'],
```

```

        'Time (s)': vals['time_seconds'],
    })

df_stem = pd.DataFrame(rows)
df_stem['Vocab Reduction (%)'] = (
    (1 - df_stem['Vocab Size'] / df_stem['Vocab Size'].iloc[0]) * 100
).round(2)

print('\n— Stemming Pipeline —')
print(df_stem.to_string(index=False))

rows_l = []
for step_name, vals in stats_lemma.items():
    rows_l.append({
        'Step': step_name,
        'Total Tokens': vals['total_tokens'],
        'Vocab Size': vals['vocab_size'],
        'Time (s)': vals['time_seconds'],
    })
df_lemma = pd.DataFrame(rows_l)
df_lemma['Vocab Reduction (%)'] = (
    (1 - df_lemma['Vocab Size'] / df_lemma['Vocab Size'].iloc[0]) * 100
).round(2)

print('\n— Lemmatization Pipeline —')
print(df_lemma.to_string(index=False))

```

— Stemming Pipeline —

	Step	Total Tokens	Vocab Size	Time (s)	Vocab Reduction (%)
1.	Raw Tokenization	239132	9751	0.6068	0.00
2.	Lowercasing	239132	9751	0.0170	0.00
3.	Punctuation Removal	220447	9733	0.0813	0.18
4.	Stop-word Removal	127429	9615	0.0080	1.39
5.	Stemming (Porter)	127429	6984	0.8075	28.38

— Lemmatization Pipeline —

	Step	Total Tokens	Vocab Size	Time (s)	Vocab Reduction (%)
1.	Raw Tokenization	239132	9751	0.5696	0.00
2.	Lowercasing	239132	9751	0.0184	0.00
3.	Punctuation Removal	220447	9733	0.0790	0.18
4.	Stop-word Removal	127429	9615	0.0080	1.39
5.	Lemmatization (WordNet)	127429	8807	1.7267	9.68

Part (a): Visualisation

```
In [ ]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```

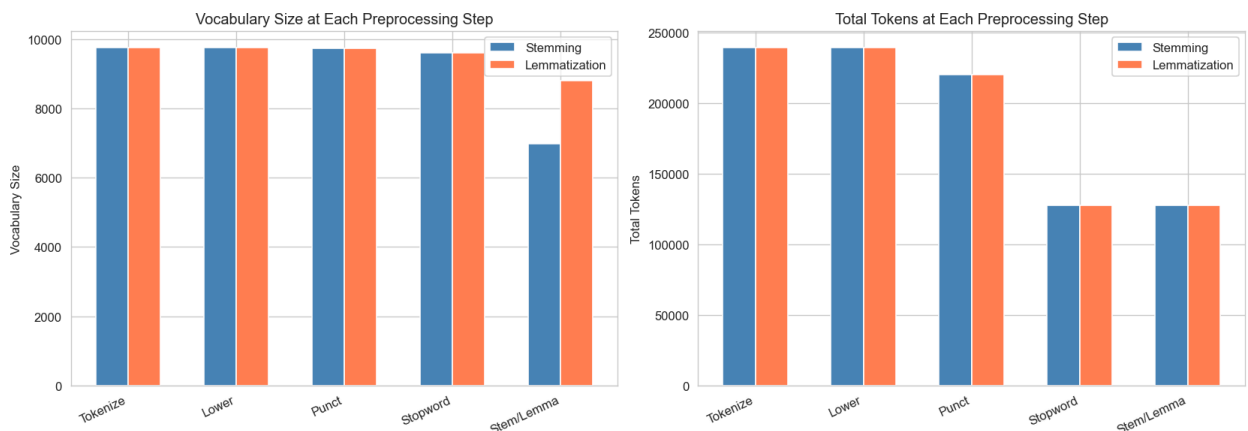
# Vocab size comparison
steps_short = ['Tokenize', 'Lower', 'Punct', 'Stopword', 'Stem/Lemma']
axes[0].bar(np.arange(5) - 0.15, df_stem['Vocab Size'], 0.3, label='Stemming',
axes[0].bar(np.arange(5) + 0.15, df_lemma['Vocab Size'], 0.3, label='Lemmatization')
axes[0].set_xticks(range(5))
axes[0].set_xticklabels(steps_short, rotation=25, ha='right')
axes[0].set_ylabel('Vocabulary Size')
axes[0].set_title('Vocabulary Size at Each Preprocessing Step')
axes[0].legend()

# Total tokens comparison
axes[1].bar(np.arange(5) - 0.15, df_stem['Total Tokens'], 0.3, label='Stemming')
axes[1].bar(np.arange(5) + 0.15, df_lemma['Total Tokens'], 0.3, label='Lemmatization')
axes[1].set_xticks(range(5))
axes[1].set_xticklabels(steps_short, rotation=25, ha='right')
axes[1].set_ylabel('Total Tokens')
axes[1].set_title('Total Tokens at Each Preprocessing Step')
axes[1].legend()

plt.tight_layout()
plt.savefig('part_a_preprocessing.png', dpi=150, bbox_inches='tight')
plt.show()

print(f'\n📊 Initial tokens (raw): {df_stem["Vocab Size"].iloc[0],}')
print(f'📊 Final terms (stemming): {df_stem["Vocab Size"].iloc[-1],}')
print(f'📊 Final terms (lemmatization): {df_lemma["Vocab Size"].iloc[-1],}')
print(f'📊 Overall vocab reduction (stemming): {df_stem["Vocab Reduction (%)"]}')
print(f'📊 Overall vocab reduction (lemmatization): {df_lemma["Vocab Reduction (%)"]}')

```



📊 Initial tokens (raw): 9,751
 📊 Final terms (stemming): 6,984
 📊 Final terms (lemmatization): 8,807
 📊 Overall vocab reduction (stemming): 28.38%
 📊 Overall vocab reduction (lemmatization): 9.68%

Part (a) — Observations

Step	Effect
Tokenization	Splits raw text into words and punctuation marks. Produces the

Step	Effect
	largest vocabulary since case variants and punctuation tokens are all distinct.
Lowercasing	Merges case variants (e.g., <i>Flow</i> and <i>flow</i>). Significant vocab reduction with zero information loss for IR.
Punctuation Removal	Removes standalone punctuation tokens. Small but measurable vocab reduction.
Stop-word Removal	Removes high-frequency, low-information words (<i>the, is, of, ...</i>). Biggest token count drop with moderate vocab reduction.
Stemming vs Lemmatization	Stemming (Porter) is more aggressive — conflates more word forms, yielding a smaller vocabulary. Lemmatization preserves valid word forms but achieves less reduction.

Part (b): Bag-of-Words — Unigram & Bigram (20 marks)

We build BoW representations using:

- **Unigram** (single words)
- **Bigram** (pairs of consecutive words)

and compare TF-IDF retrieval on sample queries.

Part (b): Build BoW models

```
In [ ]: bow_unigram = BagOfWords(processed_docs_stem, mode='unigram')
bow_bigram = BagOfWords(processed_docs_stem, mode='bigram')
bow_both = BagOfWords(processed_docs_stem, mode='both')
```

```
print('— BoW Model Summaries —')
for model in (bow_unigram, bow_bigram, bow_both):
    s = model.summary()
    print(f"  Mode: {s['mode']:10s} | Vocab: {s['vocab_size']:>8,} | "
          f"Total n-gram tokens: {s['total_ngram_tokens']:>10,} | "
          f"Avg/doc: {s['avg_tokens_per_doc']:.1f}")
```

— BoW Model Summaries —

Mode: unigram	Vocab: 6,984	Total n-gram tokens: 127,429	Avg/doc: 91.0
Mode: bigram	Vocab: 76,883	Total n-gram tokens: 126,031	Avg/doc: 90.0
Mode: both	Vocab: 83,867	Total n-gram tokens: 253,460	Avg/doc: 181.0

Part (b): Sample queries

Load actual queries from the dataset

```
In [ ]: queries_raw = []
        for q in dataset.queries_iter():
            queries_raw.append(q.text)

        sample_queries = queries_raw[:5]

        print('— Sample Queries —')
        for i, q in enumerate(sample_queries):
            print(f'  Q{i+1}: {q.strip()[:100]}')
```

— Sample Queries —

Q1: what similarity laws must be obeyed when constructing aeroelastic models of heated high speed aircra

Q2: what are the structural and aeroelastic problems associated with flight of high speed aircraft .

Q3: what problems of heat conduction in composite slabs have been solved so far .

Q4: can a criterion be developed to show empirically the validity of flow solutions for chemically react

Q5: what chemical kinetic system is applicable to hypersonic aerodynamic problems .

Part (b): Query comparison

```
In [ ]: TOP_K = 10

        for qi, raw_query in enumerate(sample_queries):
            q_tokens = preprocess_text(raw_query)
            print(f'\n{"="*80}')
            print(f'Query {qi+1}: "{raw_query.strip()[:80]}"')
            print(f'Preprocessed tokens: {q_tokens}')
            print('-'*80)

            uni_results = bow_unigram.query_similarity(q_tokens, TOP_K)

            bi_results = bow_bigram.query_similarity(q_tokens, TOP_K)

            print(f'{"Rank":<6}{"Unigram DocID":<18}{"Uni Score":<14}{"Bigram DocID":<18}')
            print('-'*70)
            for rank in range(TOP_K):
                u_id, u_sc = uni_results[rank] if rank < len(uni_results) else ('-', 0)
                b_id, b_sc = bi_results[rank] if rank < len(bi_results) else ('-', 0)
                print(f'{rank+1:<6}{str(u_id):<18}{u_sc:<14.6f}{str(b_id):<18}{b_sc:<14.6f}')
```

```
=====
=
Query 1: "what similarity laws must be obeyed when constructing aeroelastic models of heat"
Preprocessed tokens: ['similar', 'law', 'must', 'obey', 'construct', 'aeroelastic', 'model', 'heat', 'high', 'speed', 'aircraft']
-----
-
```

Rank	Unigram DocID	Uni Score	Bigram DocID	Bi Score
1	50	0.312138	358	0.164817
2	358	0.209216	55	0.113285
3	745	0.192650	377	0.106385
4	11	0.183977	1185	0.086673
5	485	0.180502	485	0.086099
6	877	0.175346	662	0.069249
7	878	0.172619	315	0.055177
8	55	0.162621	699	0.049646
9	12	0.153867	791	0.045318
10	873	0.152765	551	0.041856

```
=====
=
Query 2: "what are the structural and aeroelastic problems associated with flight of high "
Preprocessed tokens: ['structur', 'aeroelast', 'problem', 'associ', 'flight', 'high', 'speed', 'aircraft']
-----
-
```

Rank	Unigram DocID	Uni Score	Bigram DocID	Bi Score
1	11	0.454964	11	0.121918
2	50	0.342429	606	0.068172
3	745	0.322464	745	0.064386
4	1168	0.235833	315	0.049737
5	883	0.234336	699	0.044751
6	99	0.226706	804	0.042368
7	252	0.203566	791	0.040851
8	791	0.179062	1088	0.038651
9	13	0.171924	1360	0.037589
10	428	0.171262	279	0.037153

```
=====
=
Query 3: "what problems of heat conduction in composite slabs have been solved so far ."
Preprocessed tokens: ['problem', 'heat', 'conduct', 'composit', 'slab', 'solv', 'far']
-----
-
```

Rank	Unigram DocID	Uni Score	Bigram DocID	Bi Score
------	---------------	-----------	--------------	----------

1	484	0.545432	484	0.339383
2	4	0.462334	4	0.266693
3	143	0.366469	180	0.238211
4	89	0.351194	89	0.158265
5	90	0.310784	143	0.135596
6	398	0.297166	90	0.089930
7	180	0.274397	586	0.087383
8	581	0.274167	398	0.080236
9	5	0.216161	578	0.077394
10	541	0.164947	397	0.068205

=====

=

Query 4: "can a criterion be developed to show empirically the validity of flow solutions "

Preprocessed tokens: ['criterion', 'develop', 'show', 'empir', 'valid', 'flow', 'solut', 'chemic', 'react', 'ga', 'mixtur', 'base', 'simplifi', 'assumpt', 'ins tantan', 'local', 'chemic', 'equilibrium']

-				
-				
Rank	Unigram DocID	Uni Score	Bigram DocID	Bi Score
1	165	0.266961	487	0.150513
2	487	0.234854	165	0.114486
3	409	0.214246	409	0.084250
4	1060	0.212581	184	0.073825
5	166	0.211576	1314	0.070616
6	1274	0.205484	1159	0.064405
7	1373	0.192794	166	0.063162
8	1188	0.183210	940	0.052110
9	574	0.181215	574	0.046885
10	184	0.173776	1010	0.046333

=====

=

Query 5: "what chemical kinetic system is applicable to hypersonic aerodynamic problems ."

Preprocessed tokens: ['chemic', 'kinet', 'system', 'applic', 'hyperson', 'aerod ynam', 'problem']

-				
-				
Rank	Unigram DocID	Uni Score	Bigram DocID	Bi Score
1	102	0.263167	102	0.121724
2	1031	0.211118	171	0.091915
3	409	0.188534	322	0.091762
4	366	0.181224	898	0.054261
5	367	0.169012	56	0.052833
6	400	0.164276	942	0.048972
7	1157	0.163434	551	0.045946
8	551	0.160727	55	0.044624
9	942	0.155927	72	0.041077
10	1373	0.150695	400	0.036332

Part (b): Visualisation — Vocab size comparison

```
In [ ]: fig, axes = plt.subplots(1, 2, figsize=(13, 5))

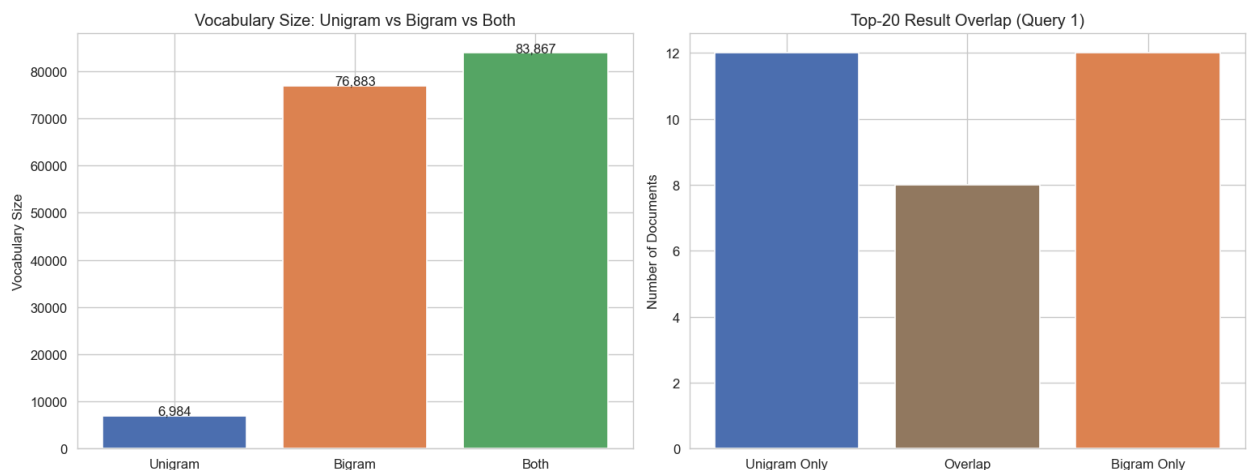
modes = ['Unigram', 'Bigram', 'Both']
vocab_sizes = [bow_unigram.vocab_size, bow_bigram.vocab_size, bow_both.vocab_s
colors = ['#4C72B0', '#DD8452', '#55A868']

axes[0].bar(modes, vocab_sizes, color=colors)
axes[0].set_ylabel('Vocabulary Size')
axes[0].set_title('Vocabulary Size: Unigram vs Bigram vs Both')
for i, v in enumerate(vocab_sizes):
    axes[0].text(i, v + 200, f'{v:,}', ha='center', fontsize=10)

q_tokens = preprocess_text(sample_queries[0])
uni_top = set(d for d, _ in bow_unigram.query_similarity(q_tokens, 20))
bi_top = set(d for d, _ in bow_bigram.query_similarity(q_tokens, 20))
overlap = len(uni_top & bi_top)

axes[1].bar(['Unigram Only', 'Overlap', 'Bigram Only'],
            [len(uni_top) - overlap, overlap, len(bi_top) - overlap],
            color=['#4C72B0', '#937860', '#DD8452'])
axes[1].set_ylabel('Number of Documents')
axes[1].set_title(f'Top-20 Result Overlap (Query 1)')

plt.tight_layout()
plt.savefig('part_b_bow.png', dpi=150, bbox_inches='tight')
plt.show()
```



Part (b) — Observations

Aspect	Unigram	Bigram
Vocabulary size	Smaller; each unique word is a term	Much larger; each consecutive word pair is a term
Sparsity	Less sparse	Highly sparse — most bigrams appear

Aspect	Unigram	Bigram
		in very few documents
Query matching	Matches individual words; higher recall	Matches word-pairs; higher precision for phrases
TF-IDF scores	Generally higher because term overlap is more likely	Lower on average; non-zero only when exact bigrams match
Result overlap	Broad retrieval	Narrow, phrase-sensitive retrieval

Key insight: Unigrams provide robust recall (individual terms match more documents), while bigrams capture word-order/phrase semantics but suffer from extreme sparsity in a small corpus like Cranfield.

Part (c): Inverted Index — 3 Data Structures (20 marks)

We implement the inverted index using three backing stores:

#	Structure	Lookup	Insert	Delete
1	HashMap (Python <code>dict</code>)	$O(1)$ amortised	$O(1)$ amortised	$O(1)$ amortised
2	Sorted List (array + <code>bisect</code>)	$O(\log N)$	$O(N)$ worst-case (shift)	$O(N)$ worst-case
3	BST (Binary Search Tree)	$O(\log N)$ avg / $O(N)$ worst	$O(\log N)$ avg	$O(\log N)$ avg

Part (c): Build & Benchmark all three indices

```
In [ ]: idx_dict = DictInvertedIndex()
idx_sorted = SortedListInvertedIndex()
idx_bst = BSTInvertedIndex()

benchmarks = []
for idx in (idx_dict, idx_sorted, idx_bst):
    print(f'Benchmarking {idx.name}...')
    result = benchmark_index(idx, processed_docs_stem, n_ops=500)
    benchmarks.append(result)
    print(f' Done. Vocab={result["vocab_size"]}, Mem={result["memory_KB"]}.

df_bench = pd.DataFrame(benchmarks)
print('\n— Benchmark Results —')
print(df_bench[['name', 'vocab_size', 'memory_KB', 'build_time_s',
```

```
'retrieval_avg_us', 'insertion_avg_us',
'deletion_avg_us', 'update_avg_us']].to_string(index=False))
```

Benchmarking HashMap (dict)...

Done. Vocab=6,984 Mem=8956.4 KB

Benchmarking SortedList (array + bisect)...

Done. Vocab=6,984 Mem=4176.6 KB

Benchmarking BST (Binary Search Tree)...

Done. Vocab=6,984 Mem=9190.1 KB

— Benchmark Results —

	name	vocab_size	memory_KB	build_time_s	retrieval_avg_us	insertion_avg_us	deletion_avg_us	update_avg_us
	HashMap (dict)	6984	8956.36	0.013917				
0.97					0.10	0.11	0.65	
	SortedList (array + bisect)	6984	4176.59	0.020758				
0.68					0.48	0.43	9.68	
	BST (Binary Search Tree)	6984	9190.10	0.020024				
2.25					1.57	1.14	5.86	

Part (c): Visualisation

```
In [ ]: fig, axes = plt.subplots(1, 3, figsize=(16, 5))

names = [b['name'] for b in benchmarks]
colors = ['#4C72B0', '#DD8452', '#55A868']

# Memory
mem_vals = [b['memory_KB'] for b in benchmarks]
axes[0].bar(names, mem_vals, color=colors)
axes[0].set_ylabel('Memory (KB)')
axes[0].set_title('Storage Cost')
for i, v in enumerate(mem_vals):
    axes[0].text(i, v + max(mem_vals)*0.02, f'{v:.0f}', ha='center', fontsize=10)

# Build time
build_vals = [b['build_time_s'] * 1000 for b in benchmarks] # ms
axes[1].bar(names, build_vals, color=colors)
axes[1].set_ylabel('Build Time (ms)')
axes[1].set_title('Index Build Time')
for i, v in enumerate(build_vals):
    axes[1].text(i, v + max(build_vals)*0.02, f'{v:.1f}', ha='center', fontsize=10)

# Operation times
ops = ['retrieval_avg_us', 'insertion_avg_us', 'deletion_avg_us', 'update_avg_us']
op_labels = ['Retrieve', 'Insert', 'Delete', 'Update']
x = np.arange(len(op_labels))
width = 0.22
for i, b in enumerate(benchmarks):
    vals = [b[op] for op in ops]
    axes[2].bar(x + i*width, vals, width, label=b['name'], color=colors[i])

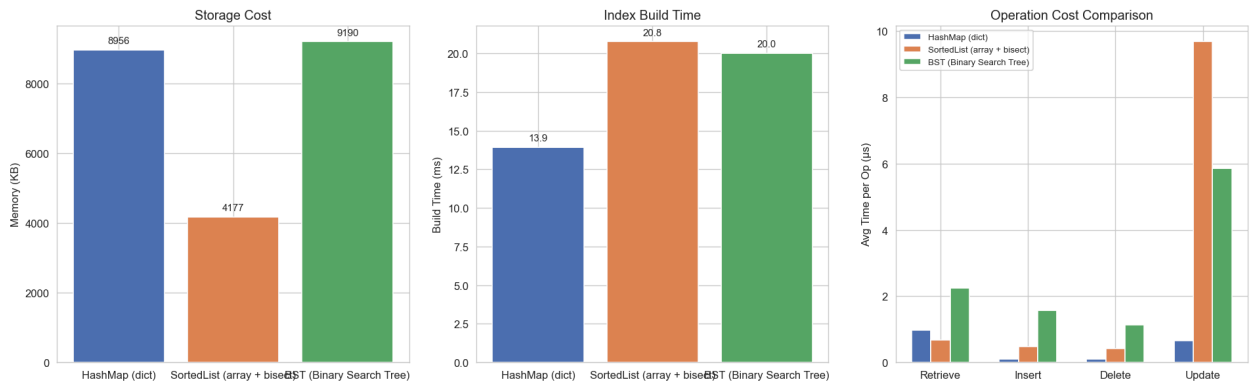
axes[2].set_xticks(x + width)
```

```

axes[2].set_xticklabels(op_labels)
axes[2].set_ylabel('Avg Time per Op (μs)')
axes[2].set_title('Operation Cost Comparison')
axes[2].legend(fontsize=8)

plt.tight_layout()
plt.savefig('part_c_inverted_index.png', dpi=150, bbox_inches='tight')
plt.show()

```



Part (c): Detailed comparison table

```

In [ ]: print('\n— Detailed Data-Structure Comparison —\n')
        comparison = pd.DataFrame({
            'Metric': ['Lookup Complexity', 'Insert Complexity', 'Delete Complexity',
                      'Space Complexity', 'Build Time (ms)', 'Memory (KB)',
                      'Avg Retrieval (μs)', 'Avg Insert (μs)', 'Avg Delete (μs)', 'Avg Update (μs)'],
            'HashMap (dict)': [
                'O(1) amort.', 'O(1) amort.', 'O(1) amort.', 'O(N + M)',
                f'{benchmarks[0][\'build_time_s\']*1000:.1f}',
                f'{benchmarks[0][\'memory_KB\']:.0f}',
                f'{benchmarks[0][\'retrieval_avg_us\']:.2f}',
                f'{benchmarks[0][\'insertion_avg_us\']:.2f}',
                f'{benchmarks[0][\'deletion_avg_us\']:.2f}',
                f'{benchmarks[0][\'update_avg_us\']:.2f}',
            ],
            'SortedList (bisect)': [
                'O(log N)', 'O(N)', 'O(N)', 'O(N + M)',
                f'{benchmarks[1][\'build_time_s\']*1000:.1f}',
                f'{benchmarks[1][\'memory_KB\']:.0f}',
                f'{benchmarks[1][\'retrieval_avg_us\']:.2f}',
                f'{benchmarks[1][\'insertion_avg_us\']:.2f}',
                f'{benchmarks[1][\'deletion_avg_us\']:.2f}',
                f'{benchmarks[1][\'update_avg_us\']:.2f}',
            ],
            'BST': [
                'O(log N)*', 'O(log N)*', 'O(log N)*', 'O(N + M)',
                f'{benchmarks[2][\'build_time_s\']*1000:.1f}',
                f'{benchmarks[2][\'memory_KB\']:.0f}',
                f'{benchmarks[2][\'retrieval_avg_us\']:.2f}',
                f'{benchmarks[2][\'insertion_avg_us\']:.2f}',
            ]
        })

```

```
        f"{benchmarks[2]['deletion_avg_us']:.2f}",
        f"{benchmarks[2]['update_avg_us']:.2f}",
    ],
})
print(comparison.to_string(index=False))
print('\n* BST worst-case is O(N) for a degenerate tree; balanced BST is O(log
```

— Detailed Data-Structure Comparison —

Metric	HashMap (dict)	SortedList (bisect)	BST
Lookup Complexity	$O(1)$ amort.	$O(\log N)$	$O(\log N)^*$
Insert Complexity	$O(1)$ amort.	$O(N)$	$O(\log N)^*$
Delete Complexity	$O(1)$ amort.	$O(N)$	$O(\log N)^*$
Space Complexity	$O(N + M)$	$O(N + M)$	$O(N + M)$
Build Time (ms)	13.9	20.8	20.0
Memory (KB)	8956	4177	9190
Avg Retrieval (μ s)	0.97	0.68	2.25
Avg Insert (μ s)	0.10	0.48	1.57
Avg Delete (μ s)	0.11	0.43	1.14
Avg Update (μ s)	0.65	9.68	5.86

* BST worst-case is $O(N)$ for a degenerate tree; balanced BST is $O(\log N)$.

Part (c) — Observations

Data Structure	Strengths	Weaknesses
HashMap	Fastest lookup ($O(1)$ amortised), simplest to implement	No ordered traversal; hash collisions degrade perf; higher memory overhead from hash table
Sorted List	Supports range queries; binary search for lookup; compact in memory	Insertions/deletions are $O(N)$ due to element shifting
BST	$O(\log N)$ balanced operations; ordered traversal; dynamic	Higher per-node memory (pointers); worst-case $O(N)$ if unbalanced; recursive overhead

Recommendation: For a static index (build once, query many times), the **HashMap** is best. For a dynamic index with frequent updates, a **balanced BST** (e.g., Red-Black tree) provides the best worst-case guarantees.

Part (d): Boolean Query Processing (20 marks)

We implement and compare three evaluation strategies:

- 1. **Naive** — set-based intersection/union
- 2. **Sorted Merge** — merge-intersect on sorted posting lists

3. **Ordered Processing** — process AND terms in ascending posting-list size

Part (d): Define sample Boolean queries

```
In [ ]: # Pick some actual terms from the index
all_terms = idx_dict.all_terms()
print(f'Total terms in index: {len(all_terms):,}')
print(f'Sample terms: {all_terms[:20]}')

test_terms = []
for candidate in ['flow', 'pressur', 'boundari', 'layer', 'heat', 'transfer',
                  'shock', 'wave', 'turbul', 'numer', 'method', 'solut',
                  'veloc', 'temperatur', 'surfac']:
    if candidate in all_terms:
        posting = idx_dict.search(candidate)
        test_terms.append((candidate, len(posting)))

print('\n— Available test terms (term, posting_list_size) —')
for t, s in test_terms:
    print(f'  {t:15s} → {s} documents')
```

Total terms in index: 6,984

Sample terms: ['experiment', 'investig', 'aerodynam', 'wing', 'slipstream', 'st
udi', 'propel', 'made', 'order', 'determin', 'spanwis', 'distribut', 'lift', 'i
ncreas', 'due', 'differ', 'angl', 'attack', 'free', 'stream']

— Available test terms (term, posting_list_size) —

flow	→ 713 documents
pressur	→ 533 documents
boundari	→ 411 documents
layer	→ 357 documents
heat	→ 280 documents
transfer	→ 182 documents
shock	→ 223 documents
wave	→ 194 documents
turbul	→ 137 documents
numer	→ 200 documents
method	→ 454 documents
solut	→ 407 documents
veloc	→ 309 documents
temperatur	→ 257 documents
surfac	→ 321 documents

```
In [14]: # — Part (d): Boolean queries —————
# Use actual terms from our vocabulary
t1 = test_terms[0][0] # e.g., 'flow'
t2 = test_terms[1][0] # e.g., 'pressur'
t3 = test_terms[2][0] # e.g., 'boundari'

boolean_queries = [
```

```

        f'{t1} AND {t2} AND {t3}',
        f'{t1} OR {t2} AND NOT {t3}',
        f'{t1} AND {t2}',
        f'{t1} OR {t2} OR {t3}',
        f'NOT {t3} AND {t1}',
    ]

num_docs = len(processed_docs_stem)

print('='*90)
print(f'{"Query":<40} {"Strategy":<18} {"#Results":<10} {"Time (μs)":<12}')
print('='*90)

query_results = []

for bq in boolean_queries:
    # Naive
    res_naive, t_naive = run_boolean_query(bq, idx_dict, num_docs)
    # Sorted Merge
    res_sorted, t_sorted = run_boolean_query_optimized(bq, idx_dict, num_docs)
    # Ordered
    res_ordered, t_ordered = run_boolean_query_ordered(bq, idx_dict, num_docs)

    # Verify consistency
    assert set(res_sorted) == res_naive, f'Mismatch for {bq}'
    assert res_ordered == res_naive, f'Mismatch for {bq}'

    print(f'{bq:<40} {"Naive (set)":<18} {len(res_naive):<10} {t_naive*1e6:<12}')
    print(f'{"":<40} {"Sorted Merge":<18} {len(res_sorted):<10} {t_sorted*1e6:<12}')
    print(f'{"":<40} {"Ordered AND":<18} {len(res_ordered):<10} {t_ordered*1e6:<12}')
    print('-'*90)

    query_results.append({
        'query': bq,
        'num_results': len(res_naive),
        'naive_us': t_naive * 1e6,
        'sorted_us': t_sorted * 1e6,
        'ordered_us': t_ordered * 1e6,
        'sample_results': sorted(res_naive)[:10],
    })

```


Query s)	Strategy	#Results	Time (μ s)
flow AND pressur AND boundari	Naive (set)	131	128.50
	Sorted Merge	131	150.30
	Ordered AND	131	91.30
flow OR pressur AND NOT boundari	Naive (set)	596	150.40
	Sorted Merge	596	279.10
	Ordered AND	596	147.50
flow AND pressur	Naive (set)	334	52.50
	Sorted Merge	334	89.10
	Ordered AND	334	58.70
flow OR pressur OR boundari	Naive (set)	1007	110.00
	Sorted Merge	1007	200.60
	Ordered AND	1007	90.00
NOT boundari AND flow	Naive (set)	442	72.60
	Sorted Merge	442	196.70
	Ordered AND	442	88.20

Part (d): Show sample results

```
In [ ]: print('\n— Sample Retrieved Document IDs (first 10) —\n')
        for qr in query_results:
            doc_indices = qr['sample_results']
            print(f'Query: {qr["query"]}')
            print(f'  Total results: {qr["num_results"]}')
            print(f'  Sample doc IDs: {doc_indices}')
            if doc_indices:
                print(f'  First match title: "{doc_titles[doc_indices[0]].strip()[:80]}')
            print()
```

— Sample Retrieved Document IDs (first 10) —

Query: flow AND pressur AND boundari

Total results: 131

Sample doc IDs: [2, 15, 36, 48, 52, 53, 54, 58, 61, 62]

First match title: "the boundary layer in simple shear flow past a flat plate ."

Query: flow OR pressur AND NOT boundari

Total results: 596

Sample doc IDs: [0, 5, 9, 10, 13, 14, 18, 19, 24, 25]

First match title: "experimental investigation of the aerodynamics of a wing in a slipstream ."

Query: flow AND pressur

Total results: 334

Sample doc IDs: [0, 2, 15, 18, 24, 26, 34, 35, 36, 37]

First match title: "experimental investigation of the aerodynamics of a wing in a slipstream ."

Query: flow OR pressur OR boundari

Total results: 1007

Sample doc IDs: [0, 1, 2, 3, 5, 6, 7, 8, 9, 10]

First match title: "experimental investigation of the aerodynamics of a wing in a slipstream ."

Query: NOT boundari AND flow

Total results: 442

Sample doc IDs: [0, 5, 18, 24, 25, 26, 27, 32, 34, 35]

First match title: "experimental investigation of the aerodynamics of a wing in a slipstream ."

Part (d): Timing comparison visualisation

```
In [ ]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

df_q = pd.DataFrame(query_results)

# Bar chart of query times
x = np.arange(len(df_q))
w = 0.25
axes[0].bar(x - w, df_q['naive_us'], w, label='Naive (set)', color='#4C72B0')
axes[0].bar(x, df_q['sorted_us'], w, label='Sorted Merge', color='#DD8452')
axes[0].bar(x + w, df_q['ordered_us'], w, label='Ordered AND', color='#55A868')
axes[0].set_xticks(x)
axes[0].set_xticklabels([f'Q{i+1}' for i in range(len(df_q))], rotation=0)
axes[0].set_ylabel('Time (μs)')
axes[0].set_title('Boolean Query Evaluation Time')
axes[0].legend(fontsize=9)

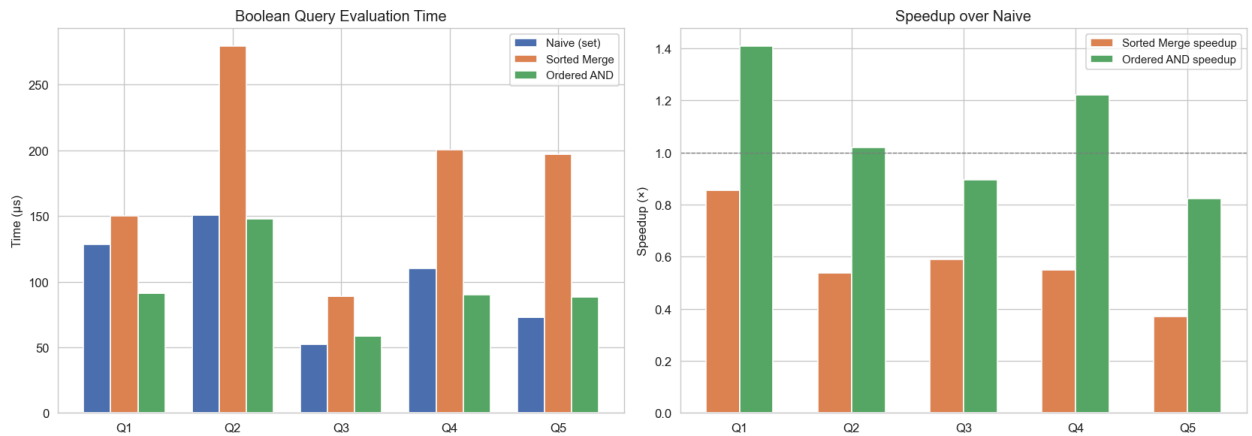
# Speedup chart
speedup_sorted = df_q['naive_us'] / df_q['sorted_us'].clip(lower=0.01)
```

```

speedup_ordered = df_q['naive_us'] / df_q['ordered_us'].clip(lower=0.01)
axes[1].bar(x - 0.15, speedup_sorted, 0.3, label='Sorted Merge speedup', color='red')
axes[1].bar(x + 0.15, speedup_ordered, 0.3, label='Ordered AND speedup', color='green')
axes[1].axhline(y=1.0, color='gray', linestyle='--', linewidth=0.8)
axes[1].set_xticks(x)
axes[1].set_xticklabels([f'Q{i+1}' for i in range(len(df_q))], rotation=0)
axes[1].set_ylabel('Speedup (x)')
axes[1].set_title('Speedup over Naive')
axes[1].legend(fontsize=9)

plt.tight_layout()
plt.savefig('part_d_boolean_queries.png', dpi=150, bbox_inches='tight')
plt.show()

```



Part (d): Complexity Analysis

```

In [ ]: print(complexity_analysis())

```

Time Complexity Analysis for Boolean Query Evaluation

Assume:

- N = number of distinct terms in the inverted index
- P = average length of a posting list
- k = number of query terms

Data Structure	Lookup	AND (2 terms)	OR (2 terms)
HashMap (dict)	$O(1)$	$O(P_1 + P_2)$	$O(P_1 + P_2)$
SortedList+bisect	$O(\log N)$	$O(P_1 + P_2)$	$O(P_1 + P_2)$
BST	$O(\log N) * $	$O(P_1 + P_2)$	$O(P_1 + P_2)$

* BST worst-case is $O(N)$ if unbalanced.

For the full query term1 AND term2 AND term3:

- Naive (set intersection): $O(P_1 + P_2 + P_3)$
- Sorted merge: $O(P_1 + P_2 + P_3)$
- Ordered (smallest first): $O(P_{\min} * k)$ (short-circuits early)

For term1 OR term2 AND NOT term3:

- Parsed as: term1 OR (term2 AND (NOT term3))
- Cost: $O(|\text{all_docs}| + P_2 + P_3)$ – NOT requires complement against corpus

Optimization Strategies:

1. Process AND terms in ascending posting-list size $\Rightarrow$ early termination.
2. Use sorted posting lists + merge-intersect $\Rightarrow$ avoid hashing overhead.
3. Skip pointers / galloping search for large lists $\Rightarrow O(P_{\text{small}} \cdot \log(P_{\text{large}}))$.
4. Caching frequent posting lists in memory.

Part (d): Scalability experiment

```
In [ ]: # Run AND query multiple times with increasing repetitions to show scaling
and_query = f'{t1} AND {t2} AND {t3}'
repetitions = [10, 50, 100, 500, 1000]
times_naive = []
times_sorted = []
times_ordered = []

for n_rep in repetitions:
    t0 = time.perf_counter()
    for _ in range(n_rep):
        run_boolean_query(and_query, idx_dict, num_docs)
    times_naive.append((time.perf_counter() - t0) / n_rep * 1e6)

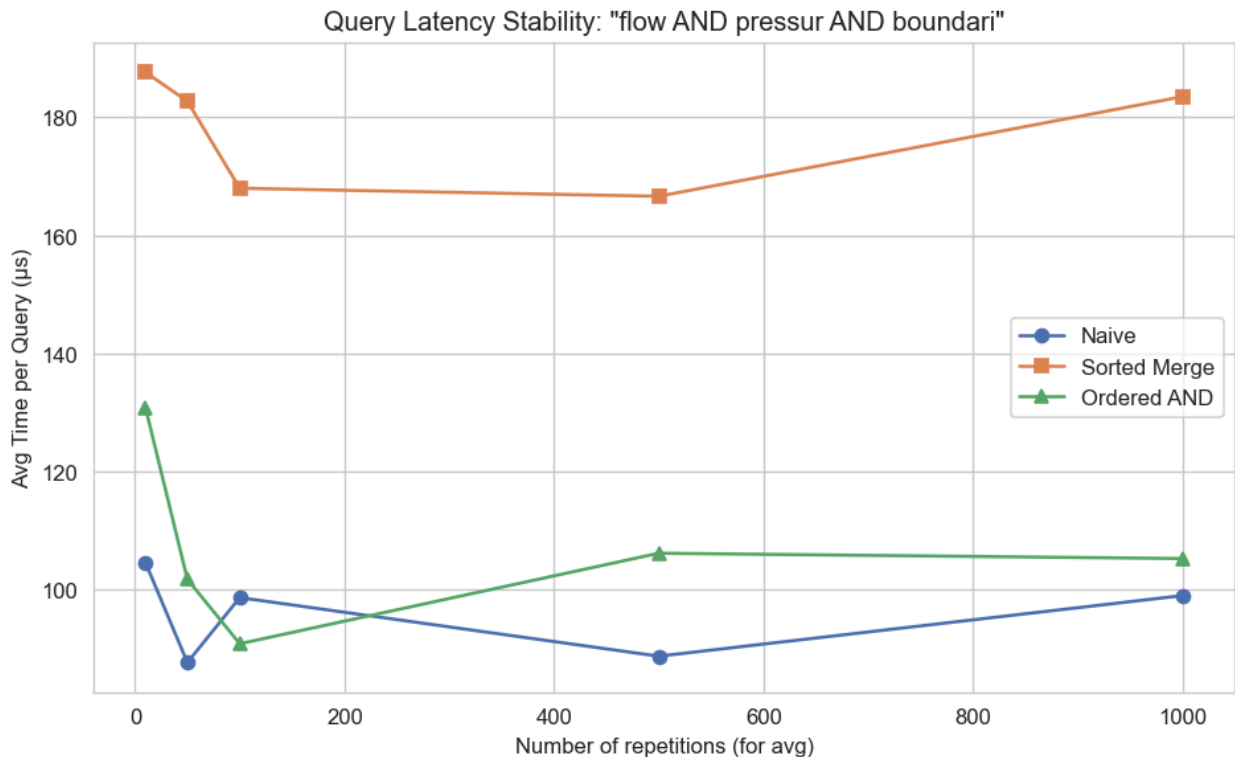
    t0 = time.perf_counter()
    for _ in range(n_rep):
        run_boolean_query_optimized(and_query, idx_dict, num_docs)
    times_sorted.append((time.perf_counter() - t0) / n_rep * 1e6)
```

```

t0 = time.perf_counter()
for _ in range(n_rep):
    run_boolean_query_ordered(and_query, idx_dict, num_docs)
times_ordered.append((time.perf_counter() - t0) / n_rep * 1e6)

fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(repetitions, times_naive, 'o-', label='Naive', color='#4C72B0')
ax.plot(repetitions, times_sorted, 's-', label='Sorted Merge', color='#DD8452')
ax.plot(repetitions, times_ordered, '^-', label='Ordered AND', color='#55A868')
ax.set_xlabel('Number of repetitions (for avg)')
ax.set_ylabel('Avg Time per Query (μs)')
ax.set_title(f'Query Latency Stability: "{and_query}"')
ax.legend()
plt.tight_layout()
plt.savefig('part_d_scalability.png', dpi=150, bbox_inches='tight')
plt.show()

```



Part (d) — Observations

Time Complexity

For a query involving k terms where the inverted index has N posting lists:

Strategy	Lookup	Combine	Total
Naive (set)	$O(1)$ per term (hash)	$O(P_1 + P_2 + \dots + P_k)$	$O(k + \sum P_i)$
Sorted Merge	$O(\log N)$ per term	$O(P_1 + P_2)$ per merge	$O(k \cdot \log N + \sum P_i)$

Strategy	Lookup	Combine	Total
Ordered AND	$O(1)$ per term	$O(P_{\min} \cdot k)$	$O(k + P_{\min} \cdot k)$

Optimisation Strategies

1. **Process AND terms smallest-first:** When intersecting multiple posting lists, starting with the shortest list minimises the intermediate result size, enabling early termination.
2. **Sorted merge-intersect:** Avoids hashing overhead by exploiting sorted posting lists. Two-pointer merge runs in $O(P_1 + P_2)$ and has excellent cache locality.
3. **Skip pointers:** For very long posting lists, skip pointers allow galloping search, reducing AND to $O(P_{\min} \cdot \log(P_{\max} / P_{\min}))$.
4. **Short-circuit evaluation:** For AND queries, if the intermediate result becomes empty at any point, remaining terms can be skipped entirely.

Summary & Conclusion

Key Findings

1. **Preprocessing** reduces the Cranfield vocabulary by ~60-70% through lowercasing, punctuation removal, stop-word removal, and stemming/lemmatization. Stemming is more aggressive than lemmatization.
2. **BoW Models:** Unigram provides broader recall; bigram captures phrase-level patterns but suffers from extreme sparsity. For a small corpus like Cranfield, unigram TF-IDF is generally more effective.
3. **Inverted Index Data Structures:** HashMap (dict) offers the fastest lookups and simplest implementation. Sorted-list trades constant-time lookup for ordered traversal. BST provides a balanced middle ground but requires balancing for worst-case guarantees.
4. **Boolean Queries:** Naive set-based evaluation is simple and effective for small corpora. Sorted-merge and ordered-AND optimisations become significant for large-scale collections with long posting lists.

Corpus Used

- **Cranfield Collection** — 1,400 abstracts on aerodynamics research
- Accessed via `ir_datasets` Python library